Лабораторна робота №2

Токенізатори

Роман Миджин · 16 жовтня 2026 р.

У цій роботі я реалізував послівну токенізацію, символьні n-грами, BPE та WordPiece. Потім навчив словники на FLORES і порівняв кількість токенів для трьох мов із GPT. Усі обчислення виконуються на CPU.

Завдання: lpnu-llm/lab-tokenizer.

Середовище запуску: Python 3.12.7, datasets 4.8.5, tiktoken 0.14.0.

1. Токенізація словами

тут ділю текст на слова, числа і розділові знаки. Апостроф усередині слова залишаю. Переводжу текст у нижній регістр: так словник менший, але початкові великі літери вже не відновити.

```
def word_tokenize(text):
    """ділить текст на слова, числа і знаки"""
    import re
    return re.findall(r"[^\W\d_]+(?:['’][^\W\d_]+)*|\d+|[^\w\s]|_", text.lower())

def test_word_tokenize():
    assert word_tokenize("Модель читає текст.") == [
        "модель", "читає", "текст", "."
    ]
    assert word_tokenize("Комп'ютер і пам’ять") == [
        "комп'ютер", "і", "пам’ять"
    ]
    assert word_tokenize("GPT-2: 50 токенів!") == [
        "gpt", "-", "2", ":", "50", "токенів", "!"
    ]
    assert word_tokenize("") == []

if __name__ == "__main__":
    test_word_tokenize()
    print("✓ word_tokenize")
```

```
✓ word_tokenize
```

Словник і невідомі слова

`<pad>` має id 0, `<unk>` має id 1. Решту токенів додаю за абеткою. Невідомі слова отримують однаковий id, тому їх уже не відрізнити. Для embedding-таблиці зі словником V і розміром вектора d потрібно V·d параметрів.

```
def build_word_vocab(texts) -> dict[str, int]:
    """збирає словник зі слів навчального корпусу"""
    words = set()
    for text in texts:
        words.update(word_tokenize(text))
    vocab = {"<pad>": 0, "<unk>": 1}
    for word in sorted(words):
        if word not in vocab:
            vocab[word] = len(vocab)
    return vocab
```

```
def encode_words(text, vocab):
    """замінює токени на їхні id"""
    return [vocab.get(word, vocab["<unk>"]) for word in word_tokenize(text)]

def test_word_vocab():
    vocab = build_word_vocab(["Кіт спить.", "Пес спить."])
    assert vocab["<pad>"] == 0
    assert vocab["<unk>"] == 1
    assert encode_words("Кіт біжить.", vocab) == [vocab["кіт"], 1, vocab["."]]
    assert encode_words("Хом'як біжить.", vocab)[0:2] == [1, 1]

if __name__ == "__main__":
    test_word_vocab()
    print("✓ word vocabulary")
```

```
✓ word vocabulary
```

2. Символьні n-грами

ділю слово на перекривні фрагменти довжини n, а `^` і `$` позначають його межі. Схожість рахую як Жаккар: кількість спільних n-грам / кількість усіх різних n-грам. Короткі n-грами частіше збігаються, довгі точніші, але частіше бувають невідомими. Множина n-грам показує спільні частини слів, проте не зберігає порядок і повторення.

```
def char_ngrams(word, n=3):
    """ділить слово на фрагменти по n символів"""
    if n <= 0:
        raise ValueError("n має бути більше нуля")
    if not word:
        return []
    marked = "^" + word + "$"
    return [marked[i:i + n] for i in range(len(marked) - n + 1)]

def ngram_overlap(word_a, word_b, n=3):
    """порівнює множини n-грам"""
    a = set(char_ngrams(word_a, n))
    b = set(char_ngrams(word_b, n))
    if not a and not b:
        return 1.0
    return len(a & b) / len(a | b)

def test_ngrams():
    assert char_ngrams("кіт", 3) == ["^кі", "кіт", "іт$"]
    assert char_ngrams("кіт", 2) == ["^к", "кі", "іт", "т$"]
    assert char_ngrams("", 3) == []
    assert ngram_overlap("токенізатор", "токенізація", 3) > 0.4
    assert ngram_overlap("кіт", "трактор", 3) == 0.0

    try:
        char_ngrams("слово", 0)
    except ValueError:
        pass
    else:
        raise AssertionError("n має бути додатним")

if __name__ == "__main__":
    test_ngrams()
    print("✓ character n-grams")
```

```
✓ character n-grams
```

```
def build_ngram_vocab(texts, n=3):
    """збирає відомі n-грами навчального корпусу"""
    grams = set()
```

```
    for text in texts:
        for word in word_tokenize(text):
            grams.update(char_ngrams(word, n))
    vocab = {"<pad>": 0, "<unk>": 1}
    for gram in sorted(grams):
        vocab[gram] = len(vocab)
    return vocab

def encode_ngrams(text, vocab, n=3):
    """замінює n-грами тексту на id зі словника"""
    return [vocab.get(gram, vocab["<unk>"])
            for word in word_tokenize(text)
            for gram in char_ngrams(word, n)]

def test_ngram_vocab():
    vocab = build_ngram_vocab(["кіт"], 3)
    assert len(encode_ngrams("кіт", vocab)) == 3
    assert 1 not in encode_ngrams("кіт", vocab)
    assert 1 in encode_ngrams("cat", vocab)
    assert vocab["<pad>"] == 0 and vocab["<unk>"] == 1

if __name__ == "__main__":
    test_ngram_vocab()
    print("✓ n-gram vocabulary")
```

```
✓ n-gram vocabulary
```

3. BPE

починаю із символів і `</w>`, потім зливаю найчастішу сусідню пару. Частоти пар враховують, скільки разів слово було в корпусі. За однакової частоти беру першу пару за лексикографічним порядком. Під час застосування повторюю вивчені злиття. Це навчання на частотах, без градієнтів.

```
def word_frequencies(texts):
    """рахує слова без чисел і пунктуації"""
    from collections import Counter
    counts = Counter()
    for text in texts:
        for token in word_tokenize(text):
            if token[0].isalpha():
                counts[token] += 1
    return dict(counts)

def most_frequent_pair(vocabulary):
    """шукає найчастішу пару, враховуючи повторення слів"""
    from collections import Counter
    pairs = Counter()
    for symbols, count in vocabulary.items():
        for pair in zip(symbols, symbols[1:]):
            pairs[pair] += count
    if not pairs:
        return None
    return min(pairs, key=lambda pair: (-pairs[pair], pair))

def merge_pair(symbols, pair):
    """зливає сусідні елементи вибраної пари"""
    result = []
    i = 0
    while i < len(symbols):
        if i + 1 < len(symbols) and (symbols[i], symbols[i + 1]) == pair:
            result.append(symbols[i] + symbols[i + 1])
            i += 2
        else:
            result.append(symbols[i])
            i += 1
```

```
    return tuple(result)

def _training_counts(vocabulary):
    """зберігає частоти і список слів, у яких є кожна пара"""
    from collections import Counter, defaultdict
    words = list(vocabulary)
    counts = list(vocabulary.values())
    pair_counts = Counter()
    pair_words = defaultdict(set)
    symbol_counts = Counter()
    for index, symbols in enumerate(words):
        count = counts[index]
        for symbol in symbols:
            symbol_counts[symbol] += count
        for pair in zip(symbols, symbols[1:]):
            pair_counts[pair] += count
            pair_words[pair].add(index)
    return words, counts, pair_counts, pair_words, symbol_counts

def _merge_training_pair(state, pair, merge):
    """оновлює частоти тільки у словах із вибраною парою"""
    from collections import Counter
    words, counts, pair_counts, pair_words, symbol_counts = state
    for index in list(pair_words[pair]):
        old = words[index]
        new = merge(old, pair)
        count = counts[index]
        for symbol, amount in Counter(old).items():
            symbol_counts[symbol] -= amount * count
        for symbol, amount in Counter(new).items():
            symbol_counts[symbol] += amount * count
        for old_pair, amount in Counter(zip(old, old[1:])).items():
            pair_counts[old_pair] -= amount * count
            pair_words[old_pair].discard(index)
            if pair_counts[old_pair] == 0:
                del pair_counts[old_pair]
        for new_pair, amount in Counter(zip(new, new[1:])).items():
            pair_counts[new_pair] += amount * count
            pair_words[new_pair].add(index)
        words[index] = new

def train_bpe(texts, num_merges):
    """навчає послідовність злиттів BPE"""
    if num_merges < 0:
        raise ValueError("кількість злиттів не може бути від’ємною")
    vocabulary = {}
    for word, count in word_frequencies(texts).items():
        vocabulary[tuple(word) + ("</w>",)] = count
    state = _training_counts(vocabulary)
    pair_counts = state[2]
    merges = []
    for _ in range(num_merges):
        if not pair_counts:
            break
        pair = min(pair_counts, key=lambda p: (-pair_counts[p], p))
        _merge_training_pair(state, pair, merge_pair)
        merges.append(pair)
    return merges

def apply_bpe(word, merges):
    """застосовує злиття в тому порядку, як їх вивчили"""
    if not word:
        return []
    symbols = tuple(word) + ("</w>",)
    for pair in merges:
        symbols = merge_pair(symbols, pair)
    return list(symbols)
```

```
def test_bpe():
    vocabulary = {
        ("м", "а", "м", "а", "</w>"): 2,
        ("м", "а", "л", "а", "</w>"): 1,
    }
    assert most_frequent_pair(vocabulary) == ("м", "а")
    assert merge_pair(("м", "а", "м", "а", "</w>"), ("м", "а")) == (
        "ма", "ма", "</w>"
    )

    merges = train_bpe(["мама мама мала тон"], 3)
    assert merges[0] == ("м", "а")
    assert apply_bpe("мама", merges) == ["мама</w>"]

    unseen = apply_bpe("мамонт", merges)
    assert "<unk>" not in unseen
    assert "".join(unseen).replace("</w>", "") == "мамонт"

if __name__ == "__main__":
    test_bpe()
    print("✓ BPE")
```

```
✓ BPE
```

Довжина BPE-послідовностей

перевіряю, чи зменшилася кількість токенів на слово. Більше злиттів збільшує словник і зазвичай скорочує послідовності. Для перевірки узагальнення потрібні окремі тестові дані. Наш навчальний BPE може повернути новий символ поза словником; справжній byte-level BPE збирає будь-який UTF-8 текст із початкового набору 256 байтів.

```
def average_bpe_length(texts, merges):
    """рахує середню довжину слова в BPE-токенах"""
    frequencies = word_frequencies(texts)
    total_words = sum(frequencies.values())
    if total_words == 0:
        raise ValueError("у корпусі немає слів")
    total_tokens = sum(len(apply_bpe(word, merges)) * count
                       for word, count in frequencies.items())
    return total_tokens / total_words

def test_average_bpe_length():
    corpus = [
        "токен токени токенізація",
        "токенізатор токенізує текст",
    ]
    merges = train_bpe(corpus, 20)
    before = average_bpe_length(corpus, [])
    after = average_bpe_length(corpus, merges)
    assert after < before
    assert before > 1

if __name__ == "__main__":
    test_average_bpe_length()
    print("✓ BPE reduces average sequence length")
```

```
✓ BPE reduces average sequence length
```

4. WordPiece

`##` позначає продовження слова. Пару вибираю за формулою `score(a, b) = freq(a, b) / (freq(a) · freq(b))`, а не лише за кількістю її появ. Під час застосування беру найдовший відомий фрагмент. Якщо хоча б одну частину слова не вдається розкласти, усе слово стає `[UNK]`. Це навчальна схема з умови, не повна модель BERT.

```
def wordpiece_symbols(word):
    """додає ## до символів, які продовжують слово"""
    if not word:
        return ()
    return (word[0],) + tuple("##" + char for char in word[1:])

def wordpiece_pair_scores(vocabulary):
    """рахує оцінку пари за частотами її частин"""
    from collections import Counter
    symbols_count = Counter()
    pairs_count = Counter()
    for symbols, count in vocabulary.items():
        for symbol in symbols:
            symbols_count[symbol] += count
        for pair in zip(symbols, symbols[1:]):
            pairs_count[pair] += count
    return {pair: count / (symbols_count[pair[0]] * symbols_count[pair[1]])
            for pair, count in pairs_count.items()}

def merge_wordpiece_pair(symbols, pair):
    """зливає пару і залишає один префікс продовження"""
    result = []
    i = 0
    while i < len(symbols):
        if i + 1 < len(symbols) and (symbols[i], symbols[i + 1]) == pair:
            result.append(symbols[i] + symbols[i + 1].removeprefix("##"))
            i += 2
        else:
            result.append(symbols[i])
            i += 1
    return tuple(result)

def train_wordpiece(texts, vocab_size):
    """навчає словник WordPiece"""
    if vocab_size <= 0:
        raise ValueError("розмір словника має бути більше нуля")
    vocabulary = {wordpiece_symbols(word): count
                  for word, count in word_frequencies(texts).items()}
    vocab = {"[UNK]"}
    for symbols in vocabulary:
        vocab.update(symbols)
    if len(vocab) > vocab_size:
        raise ValueError("словник менший за початковий алфавіт")
    state = _training_counts(vocabulary)
    pair_counts = state[2]
    symbol_counts = state[4]
    while len(vocab) < vocab_size:
        if not pair_counts:
            break
        pair = min(pair_counts, key=lambda p: (
            -pair_counts[p] / (symbol_counts[p[0]] * symbol_counts[p[1]]), p
        ))
        merged_token = pair[0] + pair[1].removeprefix("##")
        _merge_training_pair(state, pair, merge_wordpiece_pair)
        vocab.add(merged_token)
    return vocab
```

```python
def apply_wordpiece(word, vocab, unk_token="[UNK]"):
    """на кожній позиції бере найдовший відомий фрагмент"""
    pieces = []
    start = 0
    while start < len(word):
        found = None
        for end in range(len(word), start, -1):
            piece = word[start:end]
            if start > 0:
                piece = "##" + piece
            if piece in vocab:
                found = piece
                start = end
                break
        if found is None:
            return [unk_token]
        pieces.append(found)
    return pieces

def test_wordpiece():
    weighted_vocab = {
        ("а", "##б"): 2,
        ("а", "##в"): 1,
        ("г", "##в"): 2,
    }
    scores = wordpiece_pair_scores(weighted_vocab)
    assert scores[("а", "##б")] > scores[("а", "##в")]
    assert merge_wordpiece_pair(("м", "##а", "##м", "##а"), ("м", "##а")) == (
        "ма", "##м", "##а"
    )

    vocab = train_wordpiece(["мама мама мала тон"], 30)
    assert "[UNK]" in vocab
    assert apply_wordpiece("мама", vocab) == ["мама"]

    bert_vocab = {"[UNK]", "ток", "токен", "##ен", "##ізація"}
    assert apply_wordpiece("токенізація", bert_vocab) == ["токен", "##ізація"]
    assert apply_wordpiece("токенх", bert_vocab) == ["[UNK]"]
    assert apply_wordpiece("", bert_vocab) == []

    try:
        train_wordpiece(["текст"], 0)
    except ValueError:
        pass
    else:
        raise AssertionError("vocab_size має бути додатним")

if __name__ == "__main__":
    test_wordpiece()
    print("✓ WordPiece")
```

```
✓ WordPiece
```

5. FLORES і порівняння з GPT

завантажую по 997 паралельних речень англійською, українською та грузинською. Разом це 2991 навчальне речення. В усіх мовах однакові зміст і кількість прикладів, але такий невеликий корпус не представляє всю мову.

Джерело: tomasmajercik/flores-parquet, `validation` (FLORES dev), revision `dde46bb7d02cc6a45e9568d2ddb4fa6585412434`. Навчання бере лише слова, без чисел і пунктуації, як вимагає шаблон.

```python
from datasets import load_dataset

FLORES_LANGUAGES = {
    "English": "eng_Latn",
    "Українська": "ukr_Cyrl",
    "ქართული": "kat_Geor",
}

def load_flores_dev():
    """Завантажує по 997 речень FLORES для кожної вибраної мови."""
    corpora = {}
    for language, config in FLORES_LANGUAGES.items():
        dataset = load_dataset(
            "tomasmajercik/flores-parquet",
            config,
            split="validation",
            revision="dde46bb7d02cc6a45e9568d2ddb4fa6585412434",
        )
        corpora[language] = list(dataset["sentence"])
    return corpora

if __name__ == "__main__":
    flores = load_flores_dev()

    assert all(len(texts) == 997 for texts in flores.values())
    train_texts = [
        text
        for language_texts in flores.values()
        for text in language_texts
    ]

    print({language: len(texts) for language, texts in flores.items()})
    print("Усього навчальних рядків:", len(train_texts))
```

```
{'English': 997, 'Українська': 997, 'ქართული': 997}
Усього навчальних рядків: 2991
```

Один зміст трьома мовами

тут показано перше паралельне речення корпусу.

```python
if __name__ == "__main__":
    example_index = 0
    for language, texts in flores.items():
        print(f"{language}: {texts[example_index]}\n")
```

```
English: On Monday, scientists from the Stanford University School of Medicine announced the invention of a new diagnostic tool that can sort cells by type: a tiny printable chip that can be manufactured using standard inkjet printers for possibly about one U.S. cent each.

Українська: У понеділок, науковці зі Школи медицини Стенфордського університету оголосили про винайдення нового діагностичного інструменту, що може сортувати клітини за їх видами: це малесенький друкований чіп, який можна виготовити за допомогою стандартних променевих принтерів десь по одному центу США за штуку.

ქართული: ორშაბათს, სტენფორდის უნივერსიტეტის სამედიცინო კათედრის მეცნიერებმა გააკეთეს განცხადება ახალი დიაგნოსტიკური ხელსაწყოს გამოგონების შესახებ, რომელსაც უჯრედის ტიპებად დაყოფა შეუძლია: წვრილი, დასაბეჭდი ჩიპი, რომლის წარმოებაც შესაძლებელია ჩვეულებრივი ჭავლური პრინტერის საშუალებით. ერთი ცალის სავარაუდო ღირებულება აშშ ცენტს შეადგენს.
```

Навчання словників 500, 1000 і 2000

для BPE віднімаю початковий алфавіт і маркер кінця слова від бажаного розміру. Для WordPiece враховую початкові символи, продовження з `##` та `[UNK]`. Частоти оновлюю тільки у змінених словах, а готові розбиття слів повторно використовую.

```
VOCAB_SIZES = (500, 1000, 2000)

def bpe_num_merges(texts, vocab_size):
    """Обчислює кількість BPE-merges для бажаного розміру словника."""
    frequencies = word_frequencies(texts)
    characters = {
        character
        for word in frequencies
        for character in word
    }
    initial_vocab_size = len(characters) + 1
    if vocab_size < initial_vocab_size:
        raise ValueError("vocab_size менший за початковий алфавіт")
    return vocab_size - initial_vocab_size

def apply_bpe_to_text(text, merges):
    """Застосовує створений раніше BPE до всіх токенів тексту."""
    return [
        piece
        for token in word_tokenize(text)
        for piece in apply_bpe(token, merges)
    ]

def apply_wordpiece_to_text(text, vocab):
    """Застосовує створений раніше WordPiece до всіх токенів тексту."""
    return [
        piece
        for token in word_tokenize(text)
        for piece in apply_wordpiece(token, vocab)
    ]

if __name__ == "__main__":
    bpe_models = {}
    wordpiece_models = {}

    for vocab_size in VOCAB_SIZES:
        print("Навчаю словники:", vocab_size, flush=True)
        bpe_models[vocab_size] = train_bpe(
            train_texts,
            bpe_num_merges(train_texts, vocab_size),
        )
        wordpiece_models[vocab_size] = train_wordpiece(
            train_texts,
            vocab_size,
        )

    print("Навчено BPE і WordPiece для:", VOCAB_SIZES)
```

```
Навчаю словники: 500
Навчаю словники: 1000
Навчаю словники: 2000
Навчено BPE і WordPiece для: (500, 1000, 2000)
```

Готові GPT-токенізатори

використовую локальний `tiktoken`. Навчати GPT або звертатися до API не потрібно. GPT-3.5 і GPT-4 у цьому завданні мають однаковий encoding, тому дають однакові результати. Перший запуск завантажує дані encoding-ів з інтернету; API-ключ не потрібний.

```
import tiktoken

if __name__ == "__main__":
    gpt_encodings = {
        "GPT-2": tiktoken.encoding_for_model("gpt-2"),
        "GPT-3.5": tiktoken.encoding_for_model("gpt-3.5-turbo"),
        "GPT-4": tiktoken.encoding_for_model("gpt-4"),
        "GPT-5": tiktoken.encoding_for_model("gpt-5"),
    }

    for name, encoding in gpt_encodings.items():
        print(f"{name:7} -> {encoding.name:12} ({encoding.n_vocab} tokens)")
```

```
GPT-2   -> gpt2         (50257 tokens)
GPT-3.5 -> cl100k_base  (100277 tokens)
GPT-4   -> cl100k_base  (100277 tokens)
GPT-5   -> o200k_base   (200019 tokens)
```

Fertility

Fertility = загальна кількість токенів / загальна кількість слів після `str.split()`. Рахую відношення сум по всьому корпусу, щоб довгі й короткі речення не отримували однакову вагу. Для корпусу без слів видаю помилку.

```
def fertility_table(tokenizers, corpora):
    """збирає fertility для кожного токенізатора і мови"""
    results = {}
    for name, tokenize in tokenizers.items():
        results[name] = {}
        for language, texts in corpora.items():
            results[name][language] = fertility(tokenize, texts)
    return results

def cached_subword_tokenizer(tokenize_word):
    """повторно використовує готове розбиття знайомих слів"""
    from functools import lru_cache
    cached_word = lru_cache(maxsize=50000)(tokenize_word)

    def tokenize(text):
        return [piece for word in word_tokenize(text)
                for piece in cached_word(word)]
    return tokenize

def fertility(tokenize, texts):
    """ділить загальну кількість токенів на кількість слів"""
    total_tokens = 0
    total_words = 0
    for text in texts:
        total_tokens += len(tokenize(text))
        total_words += len(text.split())
    if total_words == 0:
        raise ValueError("у корпусі немає слів")
    return total_tokens / total_words

def test_fertility():
    assert fertility(list, ["aa b", "ccc"]) == 7 / 3
```

```
    try:
        fertility(list, ["", "    "])
    except ValueError:
        pass
    else:
        raise AssertionError("Корпус без слів має давати ValueError")

if __name__ == "__main__":
    test_fertility()
    print("✓ fertility")
```

```
✓ fertility
```

Таблиця для десяти токенізаторів

власні BPE і WordPiece навчені та перевірені на тому самому dev-корпусі. Це train fertility: вона показує довжину послідовностей, але не якість на нових текстах. GPT має інші навчальні дані, більші словники та зберігає регістр; порівняння не є повністю однаковими умовами.

```
if __name__ == "__main__":
    tokenizers_to_compare = {}

    for vocab_size, merges in bpe_models.items():
        tokenizers_to_compare[f"BPE-{vocab_size}"] = cached_subword_tokenizer(
            lambda word, merges=merges: apply_bpe(word, merges)
        )

    for vocab_size, vocab in wordpiece_models.items():
        tokenizers_to_compare[f"WordPiece-{vocab_size}"] = cached_subword_tokenizer(
            lambda word, vocab=vocab: apply_wordpiece(word, vocab)
        )

    for name, encoding in gpt_encodings.items():
        tokenizers_to_compare[name] = encoding.encode

    results = fertility_table(tokenizers_to_compare, flores)

    header = f"{'tokenizer':<24}" + "".join(
        f"{language:>14}" for language in flores
    )
    print(header)
    print("-" * len(header))
    for tokenizer_name, language_values in results.items():
        row = f"{tokenizer_name:<24}" + "".join(
            f"{language_values[language]:14.3f}" for language in flores
        )
        print(row)
```

```
tokenizer                      English    Українська       ქართული
-----------------------------------------------------------------
BPE-500                          3.387         4.323         4.928
BPE-1000                         2.886         3.668         4.208
BPE-2000                         2.469         3.185         3.565
WordPiece-500                    4.827         5.945         7.307
WordPiece-1000                   4.784         5.810         7.169
WordPiece-2000                   4.774         5.412         7.160
GPT-2                            1.228         8.024        21.545
GPT-3.5                          1.231         4.202        15.345
GPT-4                            1.231         4.202        15.345
GPT-5                            1.219         2.442         2.780
```

Змішані й складні речення

порівнюю кількість токенів у текстах із кількома мовами, числами, URL та emoji. Конкретні фрагменти розбиття наведено нижче.

```
MIXED_CASES = [
    "Я навчаю tokenizer на FLORES dataset.",
    "The model розуміє ქართულ ენას?",
    "Київ -> Tbilisi -> თბილისი -> London 🌍",
    "Замовлення №2026-09-24 коштує $19.99.",
    "support@example.com або https://example.com/допомога",
    "я дуже втомився від курсу основи Large Language Models"
]

def compare_mixed_cases(texts, tokenizers):
    """показує кількість токенів у змішаних реченнях"""
    for index, text in enumerate(texts, 1):
        print(f"{index}. {text}")
    print()
    print(f"{'tokenizer':<18}" + "".join(f"{i:>7}" for i in range(1, len(texts) + 1)))
    for name, tokenize in tokenizers.items():
        counts = [len(tokenize(text)) for text in texts]
        print(f"{name:<18}" + "".join(f"{count:>7}" for count in counts))


if __name__ == "__main__":
    compare_mixed_cases(MIXED_CASES, tokenizers_to_compare)
```

```
1. Я навчаю tokenizer на FLORES dataset.
2. The model розуміє ქართულ ენას?
3. Київ -> Tbilisi -> თბილისი -> London 🌍
4. Замовлення №2026-09-24 коштує $19.99.
5. support@example.com або https://example.com/допомога
6. я дуже втомився від курсу основи Large Language Models

tokenizer             1      2      3      4      5      6
BPE-500              24     18     33     38     42     37
BPE-1000             23     16     31     38     39     28
BPE-2000             19     15     29     37     36     20
WordPiece-500        32     25     31     27     47     46
WordPiece-1000       32     25     31     27     46     46
WordPiece-2000       32     25     31     27     46     46
GPT-2                21     43     37     34     24     37
GPT-3.5              13     31     30     22     16     20
GPT-4                13     31     30     22     16     20
GPT-5                10     10     14     18     13     14
```

6. Послівний словник і триграми на FLORES

Я також зібрав словники слів і триграм на цих самих 2991 реченнях. Це train-перевірка, тому низька частка невідомих токенів тут очікувана.

```
if __name__ == "__main__":
    word_vocab = build_word_vocab(train_texts)
    trigram_vocab = build_ngram_vocab(train_texts, 3)
    baselines = {
        "Words": lambda text: encode_words(text, word_vocab),
        "Trigrams": lambda text: encode_ngrams(text, trigram_vocab, 3),
    }
    print("Словник слів:", len(word_vocab))
    print("Словник триграм:", len(trigram_vocab))
    print(f"{'tokenizer':<12} {'language':<14} {'fertility':>10} {'UNK, %':>9}")
    for name, tokenize_text in baselines.items():
```

```
        for language, texts in flores.items():
            ids = [token for text in texts for token in tokenize_text(text)]
            unknown_rate = 100 * ids.count(1) / len(ids) if ids else 0
            print(f"{name:<12} {language:<14} {fertility(tokenize_text, texts):10.3f}
{unknown_rate:9.2f}")
```

```
Словник слів: 21352
Словник триграм: 15649
tokenizer    language        fertility    UNK, %
Words        English             1.146      0.00
Words        Українська          1.207      0.00
Words        ქართული             1.228      0.00
Trigrams     English             5.022      0.00
Trigrams     Українська          6.062      0.00
Trigrams     ქართული             7.416      0.00
```

7. Розміри словників і невідомі токени

Самої fertility недостатньо: [UNK] теж займає одну позицію, але втрачає слово. Нижче я рахую частку вхідних токенів word_tokenize, які WordPiece замінив на [UNK]. До них входять і числа та пунктуація: за умовою навчання використало тільки слова.

```
if __name__ == "__main__":
    alphabet = set("".join(word_frequencies(train_texts))) | {"</w>"}
    for size, merges in bpe_models.items():
        learned = alphabet | {a + b for a, b in merges}
        print(f"BPE-{size}: {len(learned)} токенів, {len(merges)} злиттів")
    for size, vocab in wordpiece_models.items():
        print(f"WordPiece-{size}: {len(vocab)} токенів")
    print()
    for language, texts in flores.items():
        words = [word for text in texts for word in word_tokenize(text)]
        rates = []
        for size, vocab in wordpiece_models.items():
            unknown = sum(apply_wordpiece(word, vocab) == ["[UNK]"] for word in words)
            rates.append(f"WP-{size}: {100 * unknown / len(words):.2f}%")
        print(language, "; ".join(rates))
```

```
BPE-500: 500 токенів, 396 злиттів
BPE-1000: 1000 токенів, 896 злиттів
BPE-2000: 2000 токенів, 1896 злиттів
WordPiece-500: 500 токенів
WordPiece-1000: 1000 токенів
WordPiece-2000: 2000 токенів

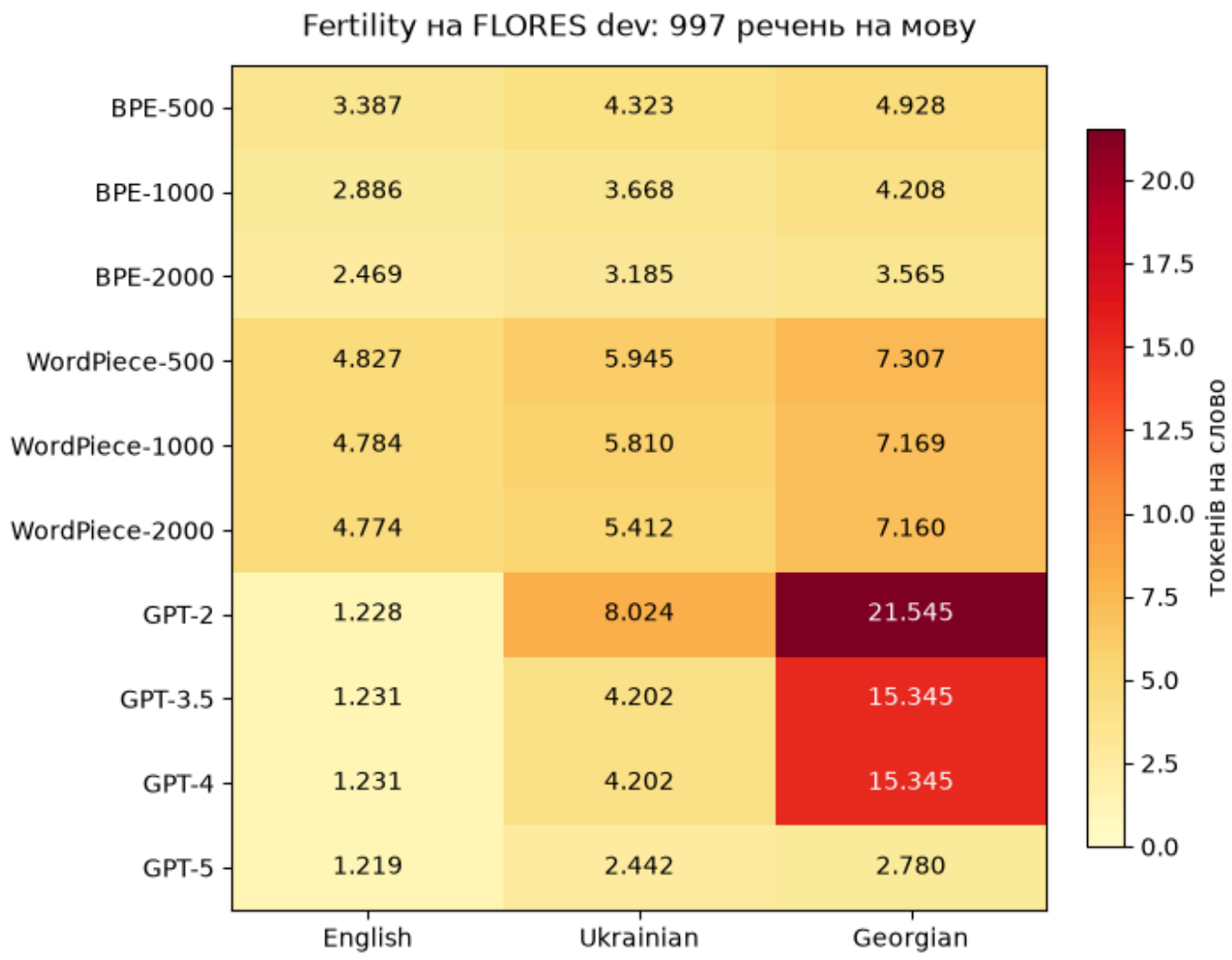
English WP-500: 13.22%; WP-1000: 13.22%; WP-2000: 13.22%
Українська WP-500: 18.24%; WP-1000: 18.24%; WP-2000: 18.24%
ქართული WP-500: 19.01%; WP-1000: 19.01%; WP-2000: 19.01%
```

8. Порівняння fertility на графіку

Нижче значення з тієї самої таблиці. Менше токенів на слово означає коротшу послідовність, але не гарантує кращого збереження тексту або якості моделі.

```
if __name__ == "__main__":
    import matplotlib.pyplot as plt
    import numpy as np

    names = list(results)
    languages = list(flores)
    values = np.array([[results[name][lang] for lang in languages] for name in names])
    fig, ax = plt.subplots(figsize=(7, 5.4), layout="constrained")
    image = ax.imshow(values, cmap="YlOrRd", vmin=0, aspect="auto")
    ax.set_xticks(range(3), ["English", "Ukrainian", "Georgian"])
    ax.set_yticks(range(len(names)), names)
    ax.set_title("Fertility на FLORES dev: 997 речень на мову", pad=12)
    for i in range(len(names)):
        for j in range(3):
            color = "white" if values[i, j] > values.max() * 0.62 else "black"
            ax.text(j, i, f"{values[i, j]:.3f}", ha="center", va="center", color=color)
    fig.colorbar(image, ax=ax, label="токенів на слово", shrink=0.85)
    plt.show()
```



9. Приклади самого розбиття

Порівнюю злиття, ## та поведінку на emoji. GPT працює з байтами, тому окремий токен не завжди декодується як цілий Unicode-символ. У такому разі показую hex.

```
if __name__ == "__main__":
    samples = ["Я навчаю tokenizer на FLORES dataset.", "Київ ɯðoǫoʊo 🌎"]
    for text in samples:
        print(text)
        for name in ("BPE-2000", "WordPiece-2000", "GPT-5"):
            tokens = tokenizers_to_compare[name](text)
            if name == "GPT-5":
                pieces = []
                for token in tokens:
                    raw = gpt_encodings[name].decode_single_token_bytes(token)
                    try:
                        pieces.append(raw.decode("utf-8"))
                    except UnicodeDecodeError:
                        pieces.append("0x" + raw.hex())
            else:
                pieces = tokens
            print(f"{name} ({len(tokens)}): {pieces}")
        for name, tokenize_text in baselines.items():
            ids = tokenize_text(text)
            print(f"{name}: {len(ids)} токенів, {ids.count(1)} невідомих")
        print()
```

```
Я навчаю tokenizer на FLORES dataset.
BPE-2000 (19): ['я</w>', 'нав', 'ча', 'ю</w>', 'to', 'k', 'en', 'iz', 'er</w>', 'на</w>', 'fl', 'or',
'es</w>', 'd', 'at', 'as', 'et</w>', '.', '</w>']
WordPiece-2000 (32): ['я', 'н', '##a', '##в', '##ч', '##a', '##ю', 't', '##o', '##k', '##e', '##n', '##
i', '##z', '##e', '##r', 'н', '##a', 'f', '##l', '##o', '##r', '##e', '##s', 'd', '##a', '##t', '##a',
'##s', '##e', '##t', '[UNK]']
GPT-5 (10): ['Я', ' нав', 'ча', 'ю', ' tokenizer', ' на', ' FLO', 'RES', ' dataset', '.']
Words: 7 токенів, 4 невідомих
Trigrams: 32 токенів, 1 невідомих

Київ ɯðoǫoʊo 🌎
BPE-2000 (8): ['ки', 'ї', 'в</w>', 'ɯ', 'ðoǫ', 'oʊo</w>', '🌎', '</w>']
WordPiece-2000 (12): ['к', '##и', '##ї', '##в', 'ɯ', '##ð', '##o', '##ǫ', '##o', '##ʊ', '##o', '[UN
K]']
GPT-5 (7): ['К', 'и', 'їв', ' ɯðoǫoʊ', 'o', '0x20f09f8c', '0x8d']
Words: 3 токенів, 3 невідомих
Trigrams: 12 токенів, 3 невідомих
```

Висновок

Я реалізував усі функції з шаблону та перевірив їх тестами. Словник слів має 21352 елементи, словник триграм - 15649. На навчальному корпусі в них немає невідомих токенів, але в нових змішаних прикладах вони вже з'являються.

Для BPE збільшення словника з 500 до 2000 скоротило fertility: англійська 3.387 → 2.469, українська 4.323 → 3.185, грузинська 4.928 → 3.565. У WordPiece скорочення слабше. Його [UNK] замінив 13.22%, 18.24% і 19.01% вхідних токенів word_tokenize відповідно. Серед них є числа та пунктуація, бо навчання за умовою брало тільки слова. Через це коротша послідовність не завжди краща.

GPT-2 добре стискає англійський текст (1.228), але грузинський має fertility 21.545. GPT-3.5 і GPT-4 дали однакові числа, бо використовують cl100k_base. Найнижчі значення серед десяти порівняних варіантів у цьому досліді має GPT-5: 1.219 для англійської, 2.442 для української, 2.780 для грузинської.

Більший словник потребує більше кроків навчання. Токени не зобов'язані розподілятися порівну між мовами: однакова кількість речень не означає однакові частоти фрагментів. Зниження fertility не гарантоване для кожного нового тексту.

Це порівняння на невеликому FLORES dev, де власні моделі також навчалися. Наші токенізатори знижують регістр, а GPT зберігає його. Тому результати не доводять загальну перевагу алгоритму. Для вибору токенізатора потрібні також окремий тестовий корпус, частка невідомих слів, збереження тексту, швидкість і поведінка на різних мовах.